

Channel与ChannelHandler

北京理工大学计算机学院
金旭亮



概述



早期的网络开发，主要是围绕着“流（Stream）”而展开的。一个Stream被定义为一个数据序列的聚合，输入流（Input Stream）用于从源读取数据，输出流（Output Stream）用于向目标写数据。



在JDK中，基于流的网络编程模型，集中于java.io包中，提供了丰富的组件和强大的功能。



虽然网络编程中用Stream进行数据传输十分方便，但是由于Stream自身的局限还是造成了其功能上的不足。例如，Stream无法实现异步操作，同时又以阻塞方式运行。另外，Stream采用单工方式，无法同时实现读／写操作。



为此，开发人员在Stream的基础上提出了Channel的概念。成为了当前网络开发的主流编程模型。

通道 (Channel) vs. 流 (Stream)

Stream

不支持异步操作，只能以阻塞方式运行。

单工方式，无法实现同时的读／写操作。

不支持Buffer（缓冲）功能，网络传输性能相对较低

Channel

完全支持异步操作，可以实现非阻塞方式运行。

全双工方式，可以同时进行读／写操作。

强制数据必须以Buffer作为容器，采用多种优化手段，提升了网络传输性能。

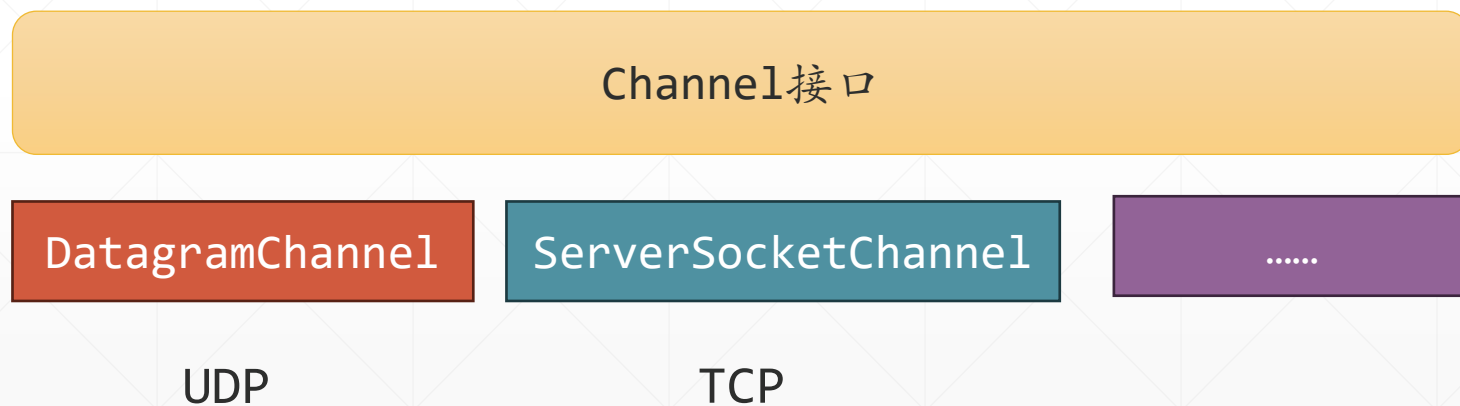


Java NIO中定义有多种Channel类型，但Netty本身提供了专门优化过的定制的Channel，本讲主要介绍Netty中的Channel。

通道 (Channel)



Channel是客户端与服务器之间的通道，客户端和服务端通过向Channel中读和写，实现数据在客户端与服务端之间的传输。Netty针对特定的应用场景，设置了不同类型的Channel类型，比如EmbeddedChannel和NioDatagramChannel。



Netty通过抽象出Channel这个接口，让网络应用可以方便地切换多种协议。

Channel是一个接口

ChannelOutboundInvoker	
close()	ChannelFuture
close(ChannelPromise)	ChannelFuture
disconnect(ChannelPromise)	ChannelFuture
voidPromise()	ChannelPromise
connect(SocketAddress, ChannelPromise)	ChannelFuture
connect(SocketAddress, SocketAddress)	ChannelFuture
write(Object, ChannelPromise)	ChannelFuture
deregister()	ChannelFuture
write(Object)	ChannelFuture
bind(SocketAddress, ChannelPromise)	ChannelFuture
deregister(ChannelPromise)	ChannelFuture
read()	ChannelOutboundInvoker
writeAndFlush(Object)	ChannelFuture
disconnect()	ChannelFuture
newProgressivePromise()	ChannelProgressivePromise
newSucceededFuture()	ChannelFuture
bind(SocketAddress)	ChannelFuture
flush()	ChannelOutboundInvoker
writeAndFlush(Object, ChannelPromise)	ChannelFuture
newPromise()	ChannelPromise
connect(SocketAddress, SocketAddress, ChannelPromise)	ChannelFuture
connect(SocketAddress)	ChannelFuture
newFailedFuture(Throwable)	ChannelFuture

Channel	
metadata()	ChannelMetadata
bytesBeforeUnwritable()	long
isRegistered()	boolean
isActive()	boolean
pipeline()	ChannelPipeline
isOpen()	boolean
closeFuture()	ChannelFuture
parent()	Channel
read()	Channel
remoteAddress()	SocketAddress
flush()	Channel
bytesBeforeWritable()	long
config()	ChannelConfig
unsafe()	Unsafe
isWritable()	boolean
id()	ChannelId
eventLoop()	EventLoop
localAddress()	SocketAddress
alloc()	ByteBufAllocator

在Netty中，为Channel接口定义了丰富的功能，让其成为整个网络编程模型的基础。左图所示为其所定义的主要成员。

Channel接口实现了读（read）、写（write）、连接（connect）和绑定（bind）等操作，还提供了Channel接口配置的功能，以及获取Channel接口对象实例事件循环（eventloop）的功能。

Channel的所有I/O操作都是异步的，表示任何I/O调用将立即返回，在返回的ChannelFuture对象实例中包含该I/O请求操作的状态信息（已成功、失败或取消）。

Channel接口生命周期



Channel接口为通道定义了四个状态：

状态	描述
ChannelUnregistered	Channel已经被创建，但还未注册到EventLoop
ChannelRegistered	Channel已经被注册到了EventLoop
ChannelActive	Channel 处于活动状态（已经连接到它的远程节点）。它现在可以接收和发送数据了
ChannelInactive	Channel没有连接到远程节点

这四个状态的切换顺序如下：



通道“注册”



问：

在前面的PPT中，我们看到有一个名为ChannelRegistered的状态，这个有何含义？

答：

通道注册这个概念，其实于Java NIO中的多路选择器（Selector）密切相关。

一个Selector，可以监控多个Socket的状态，当这些Socket“有事发生”时，会回调事先注册好的响应函数。

所谓“通道注册”，其实就是通知底层的Selector，要它关注本通道的状态，比如连接断开，有数据可读等，当状态更改时，执行一段通道事先准备好的代码。这些代码，被封装为“通道处理程序（ChannelHandler）”。

通道处理程序 (ChannelHandler)



Channel 负责传输数据，但如何处理这些数据呢？



相应的数据处理逻辑，由ChannelHandler封装。



Netty内部使用Selector监控多个通道，当通道“有事发生”，比如有数据到达，或者状态改变时，Netty负责回调ChannelHandler的相关方法。

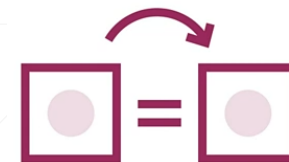


Netty提供了一些内置的ChannelHandler，在实际开发中经常使用到这些ChannelHandler，后面将结合具体应用场景，通过典型实例进行介绍。



在不影响理解的前提下，后面的课程，可能使用“Handler（处理程序）”这个术语来指代ChannelHandler。

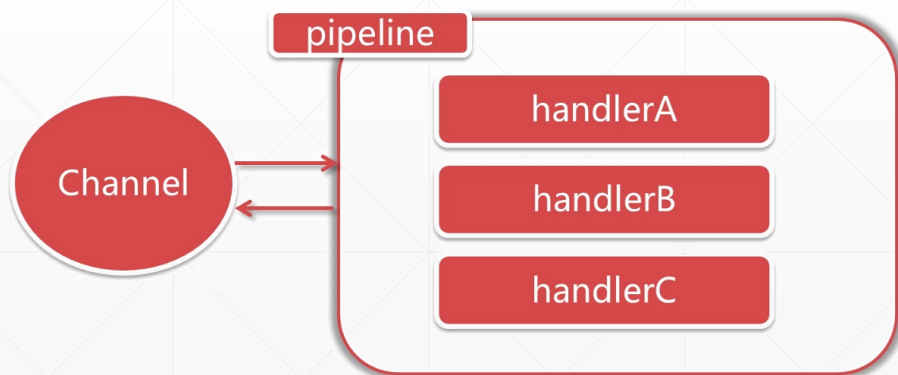
出站Handler和入站Handler



数据从外部通过Channel进入“本网络应用程序”，称为“**入站 (InBound)**”，相应地，本网络应用程序将数据发给远方的客户端，称为“**出站 (OutBound)**”。

Netty在数据“出站”和“入站”的过程中，会触发相应的事件，开发者可以为这些事件编写特定的响应对象，称为“Handler”，与Channel相关的Handler，称为“ChannelHandler”。

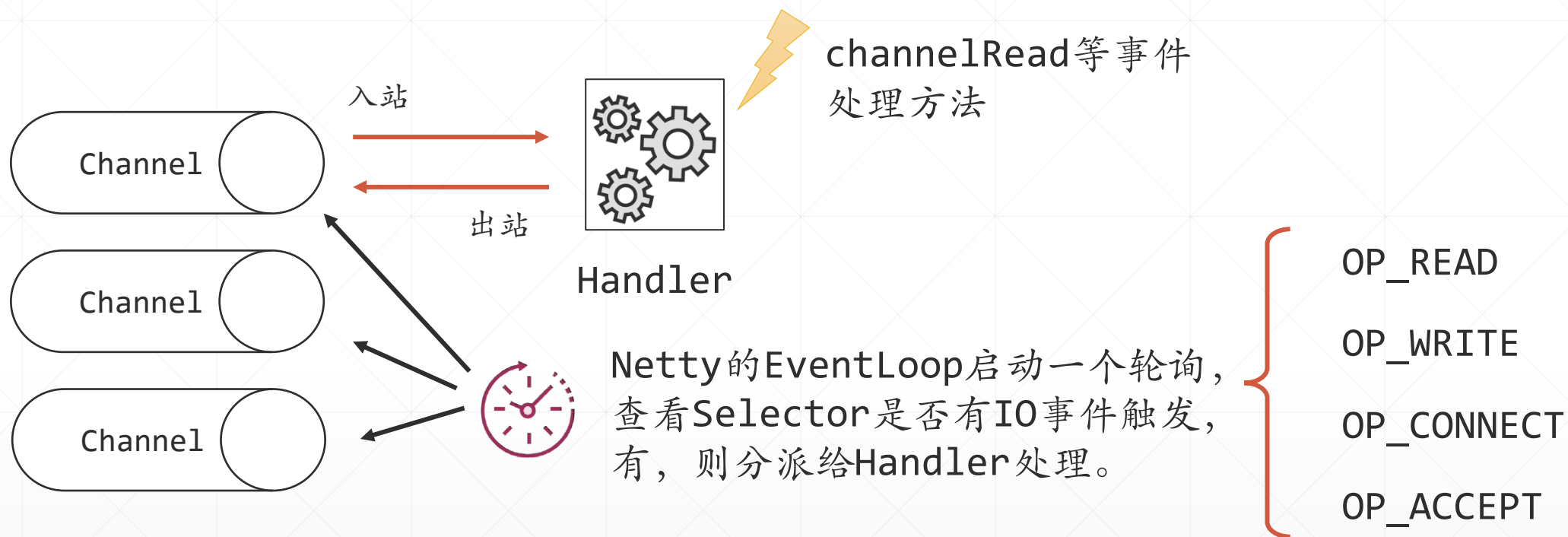
多个ChannelHandler首尾相接，构成一条“流水线 (pipeline)”，或称为“管线”，信息可以在管线中流动。入站Handler构成一条“入站管线”，对应地，出站Handler则组成一条“出站管线”。



ChannelHandler的主要职责，是对从Channel中接收到的信息进行必要的处理，之后，可以将消息发给管线中的下一个Handler，或者调用write系列方法将数据发回给客户端。

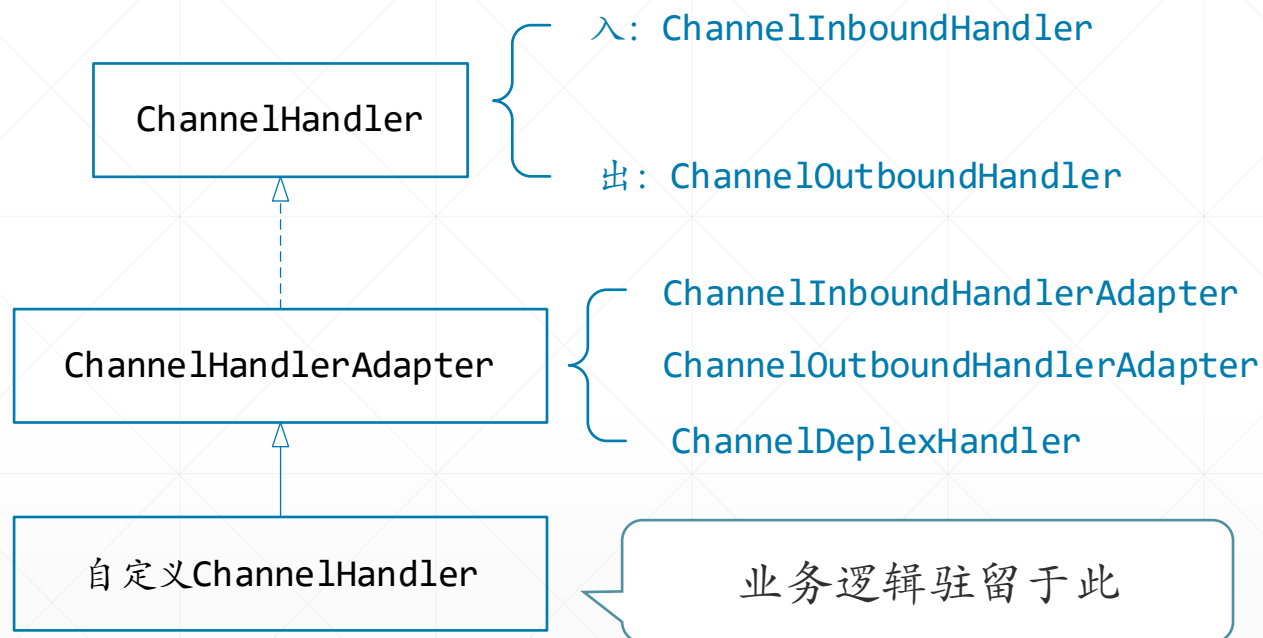
Netty提供了大量预定义的可以开箱即用的Handler组件，可以直接用来开发各种网络应用，后页将结合具体的应用场景来介绍特定的Handler。

Channel与ChannelHandler之间的配合关系



这种工作模式，类似于GUI程序中的“消息循环”与“消息队列”，仅仅只是把消息队列换成了NIO中的Selector组件。

ChannelHandler类型继承体系



Netty能区分ChannelInboundHandler实现和ChannelOutboundHandler实现,并确保数据只会在具有相同定向类型的两个ChannelHandler之间传递。

入站处理器要实现的方法

ChannelHandler		
handlerAdded	(ChannelHandlerContext)	void
exceptionCaught	(ChannelHandlerContext, Throwable)	void
handlerRemoved	(ChannelHandlerContext)	void

ChannelInboundHandler		
channelUnregistered	(ChannelHandlerContext)	void
channelInactive	(ChannelHandlerContext)	void
channelWritabilityChanged	(ChannelHandlerContext)	void
channelRead	(ChannelHandlerContext, Object)	void
userEventTriggered	(ChannelHandlerContext, Object)	void
channelRegistered	(ChannelHandlerContext)	void
channelReadComplete	(ChannelHandlerContext)	void
channelActive	(ChannelHandlerContext)	void
exceptionCaught	(ChannelHandlerContext, Throwable)	void

从上图可见，自定义入站处理器时，如果直接实现ChannelInboundHandler接口，由于此接口定义了N多的访问，是相当麻烦的，为此，Netty提供了一个ChannelInboundHandlerAdapter类，为这些方法提供了默认实现，从而简化了开发工作。

自定义客户端入站处理程序



从ChannelInboundHandlerAdapter类（或其子类）派生，重写特定的方法，是自定义入站处理程序最常见的编程方式。

```
public class MyClientSendHandler extends ChannelInboundHandlerAdapter {  
  
    @Override  
    public void channelActive(ChannelHandlerContext ctx) throws Exception {  
        System.out.println(new Date()+"客户端发送数据");  
        //将一个字符串写入到ByteBuf中  
        ByteBuf buf = ctx.alloc().buffer();  
        var messageData="你好, Netty Server!".getBytes(StandardCharsets.UTF_8);  
        buf.writeBytes(messageData);  
        //将数据发送到服务端  
        ctx.writeAndFlush(buf);  
        //信息发送完毕, 立即关闭通道  
        ctx.close();  
    }  
}
```

本讲示例中，为客户端定义了一个入站Handler，当连接上服务器后，Netty会回调它的channelActive方法，在此方法中，创建一个ByteBuf，向其中写入一条信息，再调用writeAndFlush方法发送给服务端，之后关闭通道。



对应地，从ChannelOutboundHandlerAdapter类（或其子类）派生，重写特定的方法，是自定义出站处理程序最常见的编程方式。

自定义服务端入站处理程序

在通道的整个生命周期中，会发生特定的事件，可以通过编写Handler来响应这些事件。

本讲示例编写了一个服务端入站Handler，来展示通道这些生命周期事件的触发顺序。

```
public class MyInBoundHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void handlerAdded(ChannelHandlerContext ctx) throws Exception {
        System.out.println("handlerAdded");
    }
    @Override
    public void handlerRemoved(ChannelHandlerContext ctx) throws Exception {
        System.out.println("handlerRemoved");
    }
    @Override
    public void channelRegistered(ChannelHandlerContext ctx) throws Exception {
        System.out.println("channelRegistered");
    }
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("channelActive");
    }
    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {
        System.out.println("channelRead, 收到数据: "
            + ((ByteBuf)msg).toString(StandardCharsets.UTF_8));
    }
    @Override
    public void channelReadComplete(ChannelHandlerContext ctx) throws Exception {
        System.out.println("channelReadComplete");
    }
    @Override
    public void channelInactive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("channelInactive");
    }
    @Override
    public void channelUnregistered(ChannelHandlerContext ctx) throws Exception {
        System.out.println("channelUnregistered");
    }
}
```


Handler必须办一个启用“手续”

Handler定义好之后，并不会自动“生效”，你需要将其与特定的Channel相连，或者将其并入一条ChannelPipeline。

MyNettyClient.java

```
NioEventLoopGroup workerGroup = new NioEventLoopGroup();
Bootstrap bootstrap = new Bootstrap();
bootstrap.group(workerGroup)
    .channel(NioSocketChannel.class)
    .handler(new MyClientSendHandler());
```

在Client端启用Handler，连接上服务器时，自动发送一条数据给服务器。

MyNettyServer.java

```
EventLoopGroup bossGroup = new NioEventLoopGroup();
EventLoopGroup workerGroup = new NioEventLoopGroup();

ServerBootstrap boot = new ServerBootstrap();

boot.group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new MyInBoundHandler());
```

在Server端启用Handler，展示Handler生命周期事件的触发顺序。

示例Server端运行截图

客户端发送数据



```
Run MyNettyServer x MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功
Mon Oct 07 08:19:50 CST 2024:客户端发送数据
程序退出
Process finished with exit code 0
```

服务端接收数据并打印信息



```
Run MyNettyServer x MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
handlerAdded
channelRegistered
channelActive
channelRead, 收到数据: 你好, Netty Server!
channelReadComplete
channelReadComplete
channelInactive
channelUnregistered
handlerRemoved
```